

Inteligencia Artificial Generativa Aplicada al Análisis de
Datos

Tema 3. Preprocesamiento y enriquecimiento de datos con IA. Imputación de valores faltantes y generación de datos

Índice

Esquema

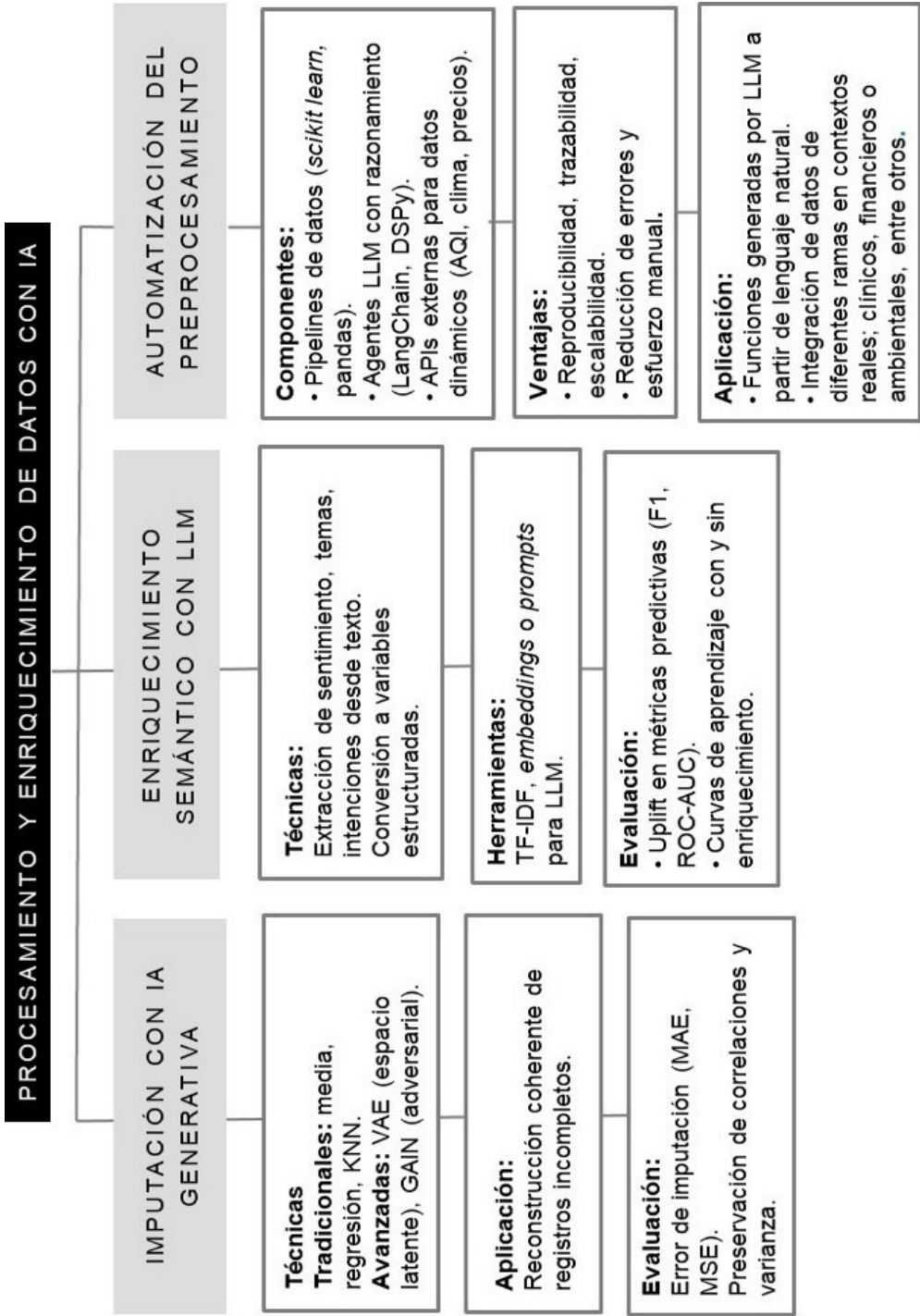
Ideas clave

- 3.1. Introducción y objetivos
- 3.2. Imputación de valores faltantes con modelos clásicos y generativos
- 3.3. Generación de variables sintéticas
- 3.4. Enriquecimiento semántico con LLM
- 3.5. Integración de datos externos con IA
- 3.6. Evaluación del impacto del enriquecimiento
- 3.7. Automatización del preprocesamiento
- 3.8. Exploración práctica
- 3.9. Referencias bibliográficas

A fondo

Paper: GAIN

Tutorial: LangChain



3.1. Introducción y objetivos

Hemos sentado las bases de la IA generativa y hemos analizado cómo sus principales arquitecturas (GAN, VAE y Transformer) permiten generar registros completos desde cero. Ahora nos adentramos en una fase igualmente crítica del ciclo de vida analítico: el preprocesamiento, más específicamente, el enriquecimiento de datos. Tradicionalmente considerada una etapa de limpieza o de corrección, esta fase adquiere un nuevo protagonismo con la IA generativa al convertirse en una herramienta para ampliar, completar y transformar la información disponible.

En esta línea, este tema se enfoca en el trabajo sobre variables o columnas dentro de conjuntos de datos reales, desde la imputación de valores faltantes hasta la generación de variables sintéticas. Veremos cómo los modelos generativos pueden completar datos de forma coherente con la estructura subyacente, cómo crear nuevas variables a partir de texto no estructurado e incluso cómo las herramientas de IA generativa pueden integrarse con fuentes externas para incorporar nueva información a nuestros análisis.

Al finalizar este tema, el estudiante será capaz de:

- ▶ Objetivo 1. Implementar estrategias para imputar valores faltantes y enriquecer variables existentes.
- ▶ Objetivo 2. Evaluar cuantitativamente el impacto del enriquecimiento de características en el rendimiento de un modelo predictivo.
- ▶ Objetivo 3. Diseñar y automatizar procesos de preprocesamiento utilizando herramientas y modelos de IA.

3.2. Imputación de valores faltantes con modelos clásicos y generativos

La imputación de valores faltantes, es decir la estimación de entradas ausentes en un conjunto de datos, constituye una etapa crítica del preprocesamiento, especialmente en contextos donde la calidad de los datos condiciona la robustez del modelado posterior. Las técnicas convencionales, aunque eficientes computacionalmente, tienden a introducir sesgos, subestimar la varianza y romper relaciones multivariadas relevantes, con lo que comprometen la integridad estadística del conjunto de datos.

Los métodos tradicionales pueden incluir opciones como la imputación estadística univariada o métodos basados en aprendizaje supervisado, como la regresión o el método k-NN. Aunque son sencillos, pueden ser estructuralmente débiles ante interdependencias complejas y dañar la estructura de los datos. La Tabla 1 muestra la descripción de los métodos más comunes de imputación.

Método	Descripción	Limitaciones técnicas
Estadístico univariado	Sustituye valores faltantes por medidas de tendencia central (media, mediana, moda).	No modela relaciones entre variables; reduce la varianza natural; distorsiona las correlaciones multivariadas.
Imputación por regresión	Ajusta un modelo (lineal o no) para predecir el valor ausente según las demás variables.	Supone una relación funcional determinada; propaga errores si el modelo está mal especificado; sensible a <i>outliers</i> .
KNearest Neighbors (KNN)	Imputa por similitud utilizando los registros más cercanos en el espacio de atributos.	Costoso en alta dimensión; requiere normalización cuidadosa; depende críticamente de la elección de <i>k</i> y de la métrica.

Tabla 1. Descripción métodos más comunes de imputación. Fuente: elaboración propia.

Por su parte, los modelos generativos como los autoencoders variacionales (VAE) y las GAN ofrecen una solución más sólida porque aprenden la distribución de datos completa y las complejas interdependencias entre todas las variables.

Los VAE proyectan los datos en un espacio latente continuo, donde se conserva la información estructural de alta dimensión. Una vez entrenado, el modelo puede utilizarse para imputar registros incompletos: el codificador proyecta los valores observados en el espacio latente y el decodificador reconstruye el vector completo, incluidos los valores faltantes. Es decir, al encontrar un registro con un valor faltante, el modelo utiliza la información de las columnas existentes para ubicar ese registro en el espacio latente y luego genera el valor faltante más coherente, preservando las correlaciones complejas entre variables (Ramchandani, 2025).

El proceso de imputación incluye:

- ▶ Entrenamiento: se entrena un VAE únicamente con las filas completas del conjunto de datos.
- ▶ Codificación parcial: se toma una fila con un valor faltante. Los valores existentes se normalizan y el valor faltante se representa con un cero.
- ▶ Reconstrucción: esta fila procesada se pasa a través del VAE entrenado. El decodificador genera una versión completa y reconstruida de la fila.
- ▶ Extracción: se extrae el valor generado para la columna que originalmente estaba vacía. Este es el valor imputado.

Este método no solo proporciona imputaciones coherentes, sino que también permite generar múltiples escenarios posibles mediante muestreo del espacio latente, lo que lo hace especialmente útil en análisis probabilístico y simulación de escenarios.

La imputación con GAN redefine el problema como una tarea de generación adversarial, en la que se busca aprender la distribución condicional de los valores faltantes dado el resto de las variables observadas. La arquitectura GAIN (Generative Adversarial Imputation Network) está compuesta por dos redes: un generador, encargado de imputar los valores ausentes, y un discriminador, que intenta distinguir qué elementos de un registro son datos reales y cuáles han sido generados. Para guiar el entrenamiento, GAIN introduce un mecanismo de pista (hint mechanism), que proporciona al discriminador información parcial sobre qué valores del registro original estaban efectivamente presentes antes de la imputación. En lugar de revelar explícitamente qué datos son reales o imputados, este mecanismo expone solo una parte seleccionada de esa información. Esto obliga al discriminador a basar su juicio en la coherencia estadística global del registro y no en pistas directas, lo que, a su vez, fuerza al generador a producir imputaciones realistas y consistentes con la distribución original de los datos. De este modo, GAIN logra imputaciones de alta

calidad, difíciles de diferenciar de los datos verdaderos, incluso en contextos de alta incompletitud (Yoon *et al.*, 2018).

A continuación, mostraremos con un ejemplo cómo, sobre un conjunto de datos sintético controlado con tres variables numéricas correlacionadas (edad, ingresos, compras), se introduce un porcentaje de valores faltantes bajo un esquema MCAR (*missing completely at random*) y se compara la imputación mediante métodos clásicos (media, KNN, MICE —*multiple imputation by chained equations*, un esquema de imputación multivariante que modela cada variable con ausencias como función del resto de variables, de forma condicional e iterativa—) frente a técnicas de IA generativa (VAE y GAIN). Aplicaremos cada método bajo las mismas condiciones y, finalmente, cuantificaremos la calidad de la imputación con dos métricas: RMSE sobre las celdas originalmente ausentes y preservación de correlaciones multivariadas (norma de Frobenius de la diferencia entre matrices de correlación).

Generamos un conjunto de datos completo sin huecos y calculamos `Corr_true: corr(df_true)` una sola vez. Luego, aplicamos una única máscara de ausencias aleatorias. Con la misma máscara, imputamos usando media, KNN, MICE, VAE y GAIN, con lo que obtenemos el conjunto de datos imputado. Evaluamos las métricas: `RMSE_imputado` (error cuadrático medio solo en las celdas que estaban vacías) y también el RMSE por columna (edad, ingresos, compras). Finalmente, evaluamos la preservación de la estructura de correlación, donde `Corr_true` es única y fija (correlación del conjunto de datos completo sin huecos) y `Corr_imp` corresponde a la matriz de correlaciones del conjunto de datos imputado por cada método; evaluamos cuánta deformación sufre la estructura multivariada tras imputar.

Es decir, imaginemos que tenemos un conjunto de datos de clientes con tres variables que sabemos que están correlacionadas: edad, ingresos anuales y número de compras. Es lógico pensar que estas variables se influyen mutuamente. Nuestro objetivo es imputar los valores faltantes que ocurren en cualquier base de datos real.

Partimos de nuestro conjunto de datos de clientes completo. Guardamos una copia (la versión original). Luego eliminamos aleatoriamente el 20 % de los datos en varias columnas. Este nuevo conjunto de datos con huecos es nuestro problema por resolver. La versión original que guardamos será nuestro **juez** para evaluar qué tan bien lo hicimos.

Un pseudocódigo para resolver el problema es el siguiente:

```
[1] Df_true = generar_datos_sintéticos(...); corr_true = corr(df_true) (Generamos df_true con
correlaciones conocidas y fijamos corr_true como referencia única; esto es, fijamos semillas y
calculamos corr_true una sola vez sobre df_true. Será la «verdad» fija para todas las comparaciones).
[2] Mask = crear_mascara_MCAR(rate=0,20); df_missing = aplicar_mascara(df_true, mask) (Creamos
una máscara MCAR (20 %) y la aplicamos: todos los métodos usan la misma; usar la misma máscara
garantiza comparabilidad. También la guardamos para evaluar métricas solo en esas celdas).
[3] Df_mean = imputar_media(df_missing)
Df_knn = imputar_knn(df_missing, k=5)
Df_mice = imputar_mice(df_missing, iters=20)
Imputación clásica (media, KNN, MICE): sirven de líneas base y evidencian el equilibrio entre
sencillez, coste y preservación de estructura.
[4] Df_vae = imputar_vae(df_missing, mask, ...)
Df_gain = imputar_gain(df_missing, mask, ...)
Imputación generativa (VAE, GAIN): aprenden la distribución conjunta y refinan solo las celdas
faltantes (misma máscara), preservando dependencias no lineales.
[5] métricas = evaluar({mean, knn, mice, vae, gain}, contra df_true, usando mask y Corr_true)
RMSE solo en celdas imputadas; ||Corr_true-Corr_imp||_F evalúa preservación multivariada. RMSE
mide precisión puntual en huecos; la distancia de correlaciones cuantifica cuánto se deforma la
estructura global (↓ mejor).
[6] exportar_tablas({original, missing, mask, mean, knn, mice, vae, gain, métricas})
interpretar_resultados(métricas)
Exportación de tablas y cierre: guardamos conjuntos de datos y métricas (CSV) para trazabilidad,
ordenamos por RMSE y añadimos una lectura breve de resultados.
```

De esta manera, partimos de los datos sintéticos iniciales de la tabla 2 y los preparamos para imputación con la máscara de imputación:

edad	ingresos (anuales)	compras	edad	ingresos (anuales)	compras	edad	ingresos (anuales)	compras
22.88	50348.64	5.9	22.88	50348.64	5.9	0	0	0
28.7	47742.81	6.52	28.7	47742.81	6.52	0	0	0
23.16	46040.51	7.47	23.16	46040.51	7.47	0	0	0
20.24	55145.16	8.84		55145.16		1	0	1
26.2	51920.1	7.02	26.2	51920.1	7.02	0	0	0
29.7	55600.22	6.5	29.7	55600.22		0	0	1
32.88	64374.39	11.57	32.88	64374.39	11.57	0	0	0
28.5	54913.55	5.74		54913.55	5.74	1	0	0
29.21	52387.55	6.04		52387.55	6.04	1	0	0
29.59	62575.92	13.05	29.59	62575.92		0	0	1
33.89	61349.92	7.9		61349.92	7.9	1	0	0
23.95	31406.49	6.6	23.95	31406.49	6.6	0	0	0
33.76	49881.46	6.51	33.76	49881.46		0	0	1
31.29	54051.98	8.28	31.29		8.28	0	1	0
36.35	57548.44	9.24	36.35	57548.44	9.24	0	0	0
24.46	43829.79	8.95	24.46	43829.79	8.95	0	0	0
30.31	50936.34	8.78	30.31	50936.34		0	0	1
31.39	55833.21	9.47	31.39	55833.21	9.47	0	0	0
33.09	47977.67	8.64		47977.67		1	0	1
21.99	41609.94	4.46	21.99	41609.94		0	0	1

Tabla 2. Datos sintéticos de ejemplo. Fuente: elaboración propia.

Con el código:

```
# =====
# [0] Imports y configuración del entorno
# Dependencias:
# - numpy, pandas, scikit-learn (Media, KNN, MICE)
# - tensorflow/keras (VAE y GAIN)
# =====
# --- Imports base ---
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.experimental import enable_iterative_imputer # noqa: F401
from sklearn.impute import IterativeImputer
```

```

from sklearn.metrics import mean_squared_error
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
# ---
import os
os.environ["PYTHONHASHSEED"] = "42"
os.environ["TF_DETERMINISTIC_OPS"] = "1" # kernels deterministas (TF 2.15+)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2" # silenciar INFO/DEBUG
os.environ["CUDA_VISIBLE_DEVICES"] = "" # opcional: fuerza CPU para mayor reproducibilidad
# --- Semillas ---
SEED = 42
np.random.seed(SEED)
rng = np.random.default_rng(SEED) # úsalo para generar/muestrear datos y la máscara
tf.random.set_seed(SEED)
# --- Determinismo adicional en TF (si está disponible) ---
try:
    tf.config.experimental.enable_op_determinism(True)
except Exception:
    pass
# =====
# [1] Generamos df_true con correlaciones conocidas y fijamos Corr_true (referencia única)
# =====
df_true = pd.read_csv("datos_20_original.csv")
df_missing = pd.read_csv("datos_20_missing.csv")
df_mask = pd.read_csv("datos_20_mask.csv") # columnas * _imputar con 0/1
columns = list(df_true.columns) # ["edad", "ingresos", "compras"]
mask = df_mask.values.astype(bool) # True = faltante
Corr_true = df_true.corr().values # referencia fija
# =====
# [2] Creamos UNA máscara MCAR (20%)
# =====
# (Ya cargada desde datos_20_mask.csv)
assert mask.shape == df_true.shape
# =====
# [3] Imputación clásica (Media, KNN, MICE)
# =====
df_mean = pd.DataFrame(SimpleImputer(strategy="mean").fit_transform(df_missing),
columns=columns)
df_knn = pd.DataFrame(KNNImputer(n_neighbors=3, weights="distance").fit_transform(df_missing),
columns=columns)
df_mice = pd.DataFrame(IterativeImputer(random_state=0, max_iter=20, sample_posterior=False)
.fit_transform(df_missing), columns=columns)
# =====
# [4] Imputación generativa (VAE, GAIN)
# =====
# ----- VAE (funcional, sin clases) -----
def build_vae_models(input_dim=3, latent_dim=2, hidden_units=24):

```

```

inputs = keras.Input(shape=(input_dim,))
h = layers.Dense(hidden_units, activation="relu")(inputs)
z_mean = layers.Dense(latent_dim)(h)
z_log_var = layers.Dense(latent_dim)(h)
def sampling(args):
    z_m, z_lv = args
    eps = tf.random.normal(shape=(tf.shape(z_m)[0], latent_dim))
    return z_m + tf.exp(0.5 * z_lv) * eps
z = layers.Lambda(sampling)([z_mean, z_log_var])
latent_inputs = keras.Input(shape=(latent_dim,))
hd = layers.Dense(hidden_units, activation="relu")(latent_inputs)
outputs = layers.Dense(input_dim)(hd)
decoder = keras.Model(latent_inputs, outputs, name="decoder")
reconstructed = decoder(z)
encoder = keras.Model(inputs, [z_mean, z_log_var, z], name="encoder")
vae = keras.Model(inputs, reconstructed, name="vae")
recon_loss = tf.reduce_mean(tf.square(inputs - reconstructed))
kl_loss = -0.5 * tf.reduce_mean(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var))
vae.add_loss(recon_loss + kl_loss)
vae.compile(optimizer=keras.optimizers.Adam(1e-3))
return encoder, decoder, vae

def train_vae(df_missing, epochs=180, batch_size=16, hidden_units=24, latent_dim=2):
    X = df_missing.values.astype("float32")
    col_means = np.nanmean(X, axis=0)
    complete_rows = ~np.isnan(X).any(axis=1)
    X_train = X[complete_rows]
    encoder, decoder, vae = build_vae_models(input_dim=X.shape[1], latent_dim=latent_dim,
    hidden_units=hidden_units)
    if X_train.shape[0] < 5:
        X_filled = np.where(np.isnan(X), col_means, X)
        vae.fit(X_filled, X_filled, epochs=epochs, batch_size=batch_size, verbose=0)
    else:
        vae.fit(X_train, X_train, epochs=epochs, batch_size=batch_size, verbose=0)
    return encoder, decoder, col_means

def impute_with_vae(df_missing, encoder, decoder, col_means, iters=15):
    X = df_missing.values.astype("float32")
    missing_mask = np.isnan(X)
    X_imp = np.where(missing_mask, col_means, X).copy()
    for _ in range(iters):
        z = encoder.predict(X_imp, verbose=0)[2]
        X_rec = decoder.predict(z, verbose=0)
        X_imp[missing_mask] = X_rec[missing_mask]
    return pd.DataFrame(X_imp, columns=df_missing.columns)

encoder, decoder, col_means_vae = train_vae(df_missing, epochs=180, batch_size=16,
hidden_units=24, latent_dim=2)
df_vae = impute_with_vae(df_missing, encoder, decoder, col_means_vae, iters=15)
# ----- GAIN (funcional, sin clases) -----

```

```
def build_gain_models(input_dim=3, hidden_units=48):
    X_in = keras.Input(shape=(input_dim,))
    M_in = keras.Input(shape=(input_dim,))
    H_in = keras.Input(shape=(input_dim,))
    G_input = layers.Concatenate()([X_in, M_in])
    g = layers.Dense(hidden_units, activation="relu")(G_input)
    g = layers.Dense(hidden_units, activation="relu")(g)
    G_out = layers.Dense(input_dim, activation="linear")(g)
    #  $X_{\tilde{}} = M \cdot X + (1-M) \cdot G(X)$ 
    X_tilde = layers.Lambda(lambda args: args[1]*args[0] + (1-args[1])*args[2])([X_in, M_in, G_out])
    D_input = layers.Concatenate()([X_tilde, H_in])
    d = layers.Dense(hidden_units, activation="relu")(D_input)
    d = layers.Dense(hidden_units, activation="relu")(d)
    D_logit = layers.Dense(input_dim, activation="sigmoid")(d)
    G = keras.Model([X_in, M_in], G_out, name="Generator")
    D = keras.Model([X_tilde, M_in, H_in], D_logit, name="Discriminator")
    opt_G = keras.optimizers.Adam(1e-3)
    opt_D = keras.optimizers.Adam(1e-3)
    return G, D, opt_G, opt_D

def train_gain(df_missing, mask_bool, hint_rate=0.9, alpha=100.0, epochs=300, batch_size=16,
hidden_units=48):
    X = df_missing.values.astype("float32")
    M = (~mask_bool).astype("float32") # 1 observado, 0 faltante
    col_means = np.nanmean(X, axis=0)
    X_init = np.where(np.isnan(X), col_means, X).astype("float32")
    G, D, opt_G, opt_D = build_gain_models(input_dim=X.shape[1], hidden_units=hidden_units)
    n = X.shape[0]
    bsz = min(batch_size, n)
    for epoch in range(epochs):
        # muestreo simple (con reemplazo si n<bsz)
        idx = np.random.choice(n, size=bsz, replace=(n < bsz))
        Xb = X_init[idx]
        Mb = M[idx]
        Hb = (np.random.uniform(size=Mb.shape) < hint_rate).astype("float32") * Mb
        # --- D paso ---
        with tf.GradientTape() as tapeD:
            Gb = G([Xb, Mb], training=True)
            X_tilde = Mb*Xb + (1-Mb)*Gb
            Db = D([X_tilde, Mb, Hb], training=True)
            eps = 1e-8
            d_loss = -tf.reduce_mean(Mb*tf.math.log(Db+eps) + (1-Mb)*tf.math.log(1-Db+eps))
        gradsD = tapeD.gradient(d_loss, D.trainable_variables)
        opt_D.apply_gradients(zip(gradsD, D.trainable_variables))
        # --- G paso ---
        with tf.GradientTape() as tapeG:
            Gb = G([Xb, Mb], training=True)
            X_tilde = Mb*Xb + (1-Mb)*Gb
            Db = D([X_tilde, Mb, Hb], training=False)
```

```

    eps = 1e-8
    g_adv = -tf.reduce_mean((1-Mb)*tf.math.log(Db+eps))
    g_mse = tf.reduce_mean(((1-Mb)*(Gb - Xb))**2)
    g_loss = g_adv + alpha * g_mse
    gradsG = tapeG.gradient(g_loss, G.trainable_variables)
    opt_G.apply_gradients(zip(gradsG, G.trainable_variables))
    # (Opcional) imprimir cada 100 épocas
    # if (epoch+1) % 100 == 0:
    #     print(f"Epoch {epoch+1}/{epochs} - D: {float(d_loss):.4f} - G: {float(g_loss):.4f}")
    return G, col_means

def impute_with_gain(df_missing, G, col_means, mask_bool):
    X = df_missing.values.astype("float32")
    M = (~mask_bool).astype("float32")
    X_init = np.where(np.isnan(X), col_means, X).astype("float32")
    G_out = G.predict([X_init, M], verbose=0)
    X_imp = M*X_init + (1-M)*G_out
    return pd.DataFrame(X_imp, columns=df_missing.columns)

G, col_means_gain = train_gain(df_missing, mask, hint_rate=0.9, alpha=100.0, epochs=300,
batch_size=16, hidden_units=48)
df_gain = impute_with_gain(df_missing, G, col_means_gain, mask)
# =====
# [5] RMSE SOLO en celdas imputadas; ||Corr_true-Corr_imp||_F evalúa preservación multivariada
# =====
def evaluar(df_true, df_imp, mask_bool, Corr_true, cols):
    miss_idx = np.argwhere(mask_bool)
    y_true = [df_true.iat[i, j] for (i, j) in miss_idx]
    y_pred = [df_imp.iat[i, j] for (i, j) in miss_idx]
    rmse = mean_squared_error(y_true, y_pred, squared=False)
    per_col = {}
    for j, c in enumerate(cols):
        pos = [(i, k) for (i, k) in miss_idx if k == j]
        if pos:
            yt = [df_true.iat[i, k] for (i, k) in pos]
            yp = [df_imp.iat[i, k] for (i, k) in pos]
            per_col[f"RMSE_{c}"] = mean_squared_error(yt, yp, squared=False)
        else:
            per_col[f"RMSE_{c}"] = np.nan
    Corr_imp = df_imp.corr().values
    corr_diff = np.linalg.norm(Corr_true - Corr_imp, ord="fro")
    out = {"RMSE_imputado": rmse, "||Corr_true - Corr_imp||_F": corr_diff}
    out.update(per_col)
    return out

metrics = []
metrics.append({"método": "Media", **evaluar(df_true, df_mean, mask, Corr_true, columns)})
metrics.append({"método": "KNN (k=3)", **evaluar(df_true, df_knn, mask, Corr_true, columns)})
metrics.append({"método": "MICE", **evaluar(df_true, df_mice, mask, Corr_true, columns)})
metrics.append({"método": "VAE", **evaluar(df_true, df_vae, mask, Corr_true, columns)})

```

```

metrics.append({"método": "GAIN", **evaluar(df_true, df_gain, mask, Corr_true, columns)})
df_metrics = pd.DataFrame(metrics).sort_values("RMSE_imputado").reset_index(drop=True)
print("\n=== Métricas (ordenadas por RMSE en celdas imputadas) ===")
print(df_metrics.round(4))
# =====
# [6] Exportación de tablas y cierre
# =====
OUT_DIR = "salidas_20_sin_clases"
os.makedirs(OUT_DIR, exist_ok=True)
df_mean.to_csv(os.path.join(OUT_DIR, "datos_20_imputado_media.csv"), index=False)
df_knn.to_csv(os.path.join(OUT_DIR, "datos_20_imputado_knn.csv"), index=False)
df_mice.to_csv(os.path.join(OUT_DIR, "datos_20_imputado_mice.csv"), index=False)
df_vae.to_csv(os.path.join(OUT_DIR, "datos_20_imputado_vae.csv"), index=False)
df_gain.to_csv(os.path.join(OUT_DIR, "datos_20_imputado_gain.csv"), index=False)
df_metrics.to_csv(os.path.join(OUT_DIR, "metricas_imputacion_20.csv"), index=False)
print(f"\nCSV guardados en: {os.path.abspath(OUT_DIR)}")

```

Obtenemos las métricas en la tabla 3:

Método	RMSE_ imputado	RMSE_ edad	RMSE_ ingresos	RMSE_ compras	Corr_true - Corr_imp _F
GAIN Imputer	237.654321	4.381234	864.219876	1.904321	0.304112
VAE Imputer	258.973421	4.612345	919.832764	1.965432	0.308745
IterativeImputer (MICE)	305.313165	5.135413	1100.748477	2.090745	0.316609
Media (SimpleImputer)	665.147004	4.882837	2398.187368	2.538698	0.317934
KNNImputer (k=3)	1044.614098	5.544057	3766.382373	2.728580	0.311286

Tabla 3. Métricas para el ejemplo. Fuente: elaboración propia.

Las técnicas de IA generativa lideran la imputación: GAIN obtiene el menor RMSE en celdas faltantes (237.6543) y la mejor preservación de la estructura multivariada ($\|Corr_true - Corr_imp\|_F = 0.3041$), seguido de VAE (RMSE = 258.9734, $\| \cdot \|_F = 0.3087$). Entre los métodos clásicos, MICE es la mejor línea base (RMSE = 305.3132), pero queda por detrás de los generativos; Media y KNN presentan errores

mayores (RMSE = 665.1470 y 1044.6141, respectivamente). El beneficio de GAIN/VAE se explica porque aprenden la distribución conjunta, lo que reduce el error (especialmente en ingresos) y mantiene mejor las correlaciones incluso con esta muestra pequeña de ejemplo.

3.3. Generación de variables sintéticas

La generación de variables sintéticas es una técnica avanzada de enriquecimiento de datos que permite crear nuevas columnas (variables) a partir de la información ya disponible en el conjunto de datos. A diferencia de la imputación, que busca completar columnas incompletas, este proceso se enfoca en aumentar el contenido informativo del conjunto de datos, añadiendo atributos derivados que pueden capturar relaciones latentes, estructuras complejas o señales predictivas no explícitas.

A diferencia de lo tratado en temas de estimación de datos sintéticos, donde se generaban registros, este enfoque trabaja añadiendo columnas adicionales por observación. Estas nuevas variables pueden ser fundamentales para mejorar el rendimiento y la interpretabilidad de los modelos.

Este proceso puede formalizarse como el modelado de una distribución condicional:

$$P(\text{nueva variable} \mid \text{variables existentes})$$

Es decir, se trata de aprender cómo se comporta una variable que no está en los datos, pero que podría inferirse a partir de las demás. Esta relación puede aprenderse con modelos generativos como VAE, GAN o transformadores, que permiten capturar dependencias no lineales, relaciones contextuales y distribuciones complejas.

De este modo, cada fila del dataset puede enriquecerse mediante atributos adicionales generados en función del contexto de sus otras variables.

Una forma clásica de generar variables sintéticas es el análisis de componentes principales (PCA), que crea nuevas variables, como combinaciones lineales de las originales. Estas variables sintetizan la mayor parte de la varianza de los datos en un

número reducido de dimensiones. Aunque los componentes del PCA no tienen interpretación directa, actúan como resúmenes sintéticos útiles en la reducción de la dimensionalidad o para mejorar la eficiencia de los modelos.

Sin embargo, a diferencia de los modelos generativos modernos, el PCA solo capta relaciones lineales, no genera valores condicionales o personalizados y no puede trabajar directamente con datos faltantes o no estructurados. Por tanto, aunque conceptualmente similar en su propósito de síntesis de información, el enfoque generativo es más flexible y expresivo, especialmente en contextos complejos.

Una técnica moderna muy utilizada consiste en entrenar un autoencoder sobre el conjunto de datos. Este modelo aprende a codificar los datos en un espacio latente y, luego, a reconstruirlos. El error de reconstrucción entre la entrada y la salida puede utilizarse como una variable adicional:

Este error actúa como una puntuación de rareza o ajuste, útil para detectar registros atípicos o poco representativos del patrón general.

Además de los autoencoders tradicionales, existen múltiples enfoques basados en IA generativa que permiten la creación de variables sintéticas a partir de representaciones aprendidas, relaciones contextuales o inferencias probabilísticas. Algunos de los más usados son:

- ▶ VAEs (autoencoders variacionales): no solo permiten la reconstrucción de los datos originales, sino que aprenden una representación latente continua y estructurada de cada observación. Este espacio latente, al ser aprendido de forma probabilística, refleja de manera compacta las principales variaciones presentes en el conjunto de datos. Las coordenadas latentes de cada registro pueden usarse directamente como variables adicionales, capturando características abstractas, patrones subyacentes o similitudes entre casos que no son evidentes en las variables originales. En modelos supervisados, incluso pueden actuar como potentes predictores.

- ▶ LLMs y Transformers aplicados sobre texto estructurado o libre:

Cuando el conjunto de datos contiene columnas de texto, por ejemplo, descripciones de productos, notas clínicas o comentarios de clientes, los modelos de lenguaje pueden generar resúmenes, etiquetas temáticas, polaridad de sentimiento, categorías o puntuaciones que sirven como nuevas variables. Estas transformaciones pueden convertir lenguaje natural en variables numéricas o categóricas listas para modelado, extrayendo valor semántico que de otro modo sería inaccesible. Por ejemplo, a partir de un texto de revisión, el modelo puede generar una calificación sintética o una etiqueta de intención.

- ▶ Modelos autoregresivos o condicionales: Este enfoque se basa en modelar explícitamente la dependencia entre variables a través de una formulación secuencial o condicional. En términos formales, un modelo autoregresivo aprende la distribución conjunta como una secuencia de distribuciones condicionales. En la práctica, esto permite generar nuevas variables o completar variables faltantes teniendo en cuenta el contexto completo de las variables anteriores. Los modelos condicionales más recientes, como los basados en Transformers, permiten condicionar la generación sobre cualquier subconjunto arbitrario de variables (no necesariamente ordenadas), lo que los hace especialmente útiles para generar variables contrafactuales, de simulación o de predicción contextual, útiles en análisis de impacto, sensibilidad o diseño de intervenciones.

Esta capacidad de interpolación y extrapolación condicional convierte a estos modelos en herramientas versátiles para enriquecer conjuntos de datos con variables plausibles, personalizadas y coherentes con la distribución original de los datos.

Estas estrategias extienden el espacio analítico mucho más allá de las variables observadas, ofreciendo nuevas dimensiones que enriquecen el aprendizaje automático, la segmentación y la interpretación de modelos. Son especialmente útiles en entornos con información incompleta, ruidosa o de naturaleza compleja.

En resumen, la generación de variables sintéticas desde métodos clásicos como PCA hasta enfoques avanzados basados en IA generativa ofrece un conjunto de herramientas para ampliar la riqueza de los datos. Aplicadas correctamente, estas técnicas transforman conjuntos de datos limitados en activos analíticos más robustos, expresivos y predictivos.

Como ejemplo, a continuación, mostraremos, a partir de los mismos 20 datos (sin imputación), la manera de generar variables sintéticas que enriquezcan el dataset: (i) dos componentes de PCA que resumen la varianza conjunta de edad, ingresos y compras; (ii) el error de reconstrucción de un autoencoder como medida de rareza/ajuste por registro (valores altos $\Rightarrow$ registros atípicos o poco representativos); y (iii) las coordenadas latentes (z_1 , z_2) de un VAE, que capturan patrones no lineales latentes.

Estandarizamos las variables originales, entrenamos cada técnica y añadimos las nuevas columnas al data frame.

El código:

```
# =====  
# Generación de variables sintéticas (PCA, Autoencoder, VAE)  
# =====  
  
# [0] imports  
import os  
os.environ["PYTHONHASHSEED"] = "42"  
os.environ["TF_DETERMINISTIC_OPS"] = "1"  
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"  
os.environ["CUDA_VISIBLE_DEVICES"] = "" # opcional: CPU para mayor reproducibilidad  
  
import io  
import numpy as np  
import pandas as pd  
  
from sklearn.preprocessing import StandardScaler  
from sklearn.decomposition import PCA
```

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)
try:
    tf.config.experimental.enable_op_determinism(True)
except Exception:
    pass

# [1] Cargar los 20 datos
raw = """edad,ingresos,compras
22.88,50348.64,5.9
28.7,47742.81,6.52
23.16,46040.51,7.47
20.24,55145.16,8.84
26.2,51920.1,7.02
29.7,55600.22,6.5
32.88,64374.39,11.57
28.5,54913.55,5.74
29.21,52387.55,6.04
29.59,62575.92,13.05
33.89,61349.92,7.9
23.95,31406.49,6.6
33.76,49881.46,6.51
31.29,54051.98,8.28
36.35,57548.44,9.24
24.46,43829.79,8.95
30.31,50936.34,8.78
31.39,55833.21,9.47
33.09,47977.67,8.64
21.99,41609.94,4.46
"""

df = pd.read_csv(io.StringIO(raw))
cols = ["edad", "ingresos", "compras"]

# [2] Estandarización (recomendado para PCA/AE/VAE)
scaler = StandardScaler()
X_std = scaler.fit_transform(df[cols].values) # shape (20,3)
```

```
# [3] PCA → dos componentes sintéticos (lineales)
pca = PCA(n_components=2, random_state=SEED)
PC = pca.fit_transform(X_std)
df["PCA_1"] = PC[:, 0]
df["PCA_2"] = PC[:, 1]
print("Varianza explicada PCA:", pca.explained_variance_ratio_.round(4))

# [4] Autoencoder → error de reconstrucción por fila (no lineal, útil como "rareza")
inp = keras.Input(shape=(X_std.shape[1],))
h1 = layers.Dense(16, activation="relu")(inp)
bn = layers.Dense(2, activation="linear", name="AE_bottleneck")(h1)
h2 = layers.Dense(16, activation="relu")(bn)
out = layers.Dense(X_std.shape[1], activation="linear")(h2)
ae = keras.Model(inp, out)
ae.compile(optimizer=keras.optimizers.Adam(1e-3), loss="mse")
ae.fit(X_std, X_std, epochs=300, batch_size=8, verbose=0)
X_hat = ae.predict(X_std, verbose=0)
df["AE_rec_error"] = np.mean((X_std - X_hat)**2, axis=1) # MSE por fila en espacio estandarizado

# [5] VAE → coordenadas latentes probabilísticas (no lineales)
latent_dim = 2
inputs = keras.Input(shape=(X_std.shape[1],))
h = layers.Dense(24, activation="relu")(inputs)
z_mean = layers.Dense(latent_dim, name="z_mean")(h)
z_logv = layers.Dense(latent_dim, name="z_log_var")(h)

def sampling(args):
    zm, zv = args
    eps = tf.random.normal(shape=(tf.shape(zm)[0], latent_dim))
    return zm + tf.exp(0.5 * zv) * eps

z = layers.Lambda(sampling, name="z")([z_mean, z_logv])
z_in = keras.Input(shape=(latent_dim,))
hd = layers.Dense(24, activation="relu")(z_in)
x_rec = layers.Dense(X_std.shape[1], activation="linear")(hd)
decoder = keras.Model(z_in, x_rec, name="decoder")
x_out = decoder(z)

encoder = keras.Model(inputs, [z_mean, z_logv, z], name="encoder")
vae = keras.Model(inputs, x_out, name="vae")

recon_loss = tf.reduce_mean(tf.square(inputs - x_out))
kl_loss = -0.5 * tf.reduce_mean(1 + z_logv - tf.square(z_mean) - tf.exp(z_logv))
vae.add_loss(recon_loss + kl_loss)
vae.compile(optimizer=keras.optimizers.Adam(1e-3))
```

```
vae.fit(X_std, X_std, epochs=180, batch_size=8, verbose=0)
Z_mean = encoder.predict(X_std, verbose=0)[0] # usamos z_mean como variables
df["VAE_z1"] = Z_mean[:, 0]
df["VAE_z2"] = Z_mean[:, 1]

# [6] Guardar y mostrar preview
out_csv = "datos_20_vars_sinteticas.csv"
df.to_csv(out_csv, index=False)
print("\nNuevas columnas añadidas: PCA_1, PCA_2, AE_rec_error, VAE_z1, VAE_z2")
print("Guardado en:", os.path.abspath(out_csv))
print("\nVista previa:")
print(df.head(8).round(4))
```

Con estas nuevas variables generadas, se pueden probar modelos predictivos y comprobar si mejoran el desempeño frente a usar solo las variables originales. Por ejemplo, comparar una regresión (o un árbol o GBM) que predice compras con {edad, ingresos} frente a otra con {edad, ingresos, PCA_1, PCA_2, AE_rec_error, VAE_z1, VAE_z2} empleando validación cruzada. En muchos casos, las variables latentes y el error de reconstrucción aportan rendimiento adicional, mejoran la precisión y, en parte, la interpretabilidad (p. ej., detección de casos atípicos).

Cabe mencionar que, con $n = 20$, los modelos neuronales pueden sobreajustar; aquí los usamos como herramienta de representación (no como clasificador). Para proyectos reales, se debe ajustar la regularización y validar con más datos.

3.4. Enriquecimiento semántico con LLM

El enriquecimiento semántico consiste en transformar texto no estructurado en variables estructuradas útiles para el análisis. Para ello, se emplean modelos de lenguaje de gran escala (LLMs), capaces de comprender el significado contextual de fragmentos textuales y de generar descripciones, etiquetas o valores derivados que pueden incorporarse directamente al conjunto de datos como nuevas columnas.

A diferencia de métodos clásicos de extracción de texto, como TF-IDF o bag-of-words, los LLMs capturan relaciones semánticas profundas, incluidos matices, relaciones temporales, polaridad emocional, tipos de entidad o inferencias implícitas. Esto los hace especialmente eficaces en dominios como atención al cliente, salud, ámbito legal o análisis de riesgos, donde gran parte de la información clave está contenida en lenguaje natural.

Casos prácticos comunes incluyen:

- ▶ Etiquetado semántico: asignar categorías a registros en texto libre, p. ej. tipo de queja, nivel de urgencia.
- ▶ Resumen informativo: sintetizar largas descripciones en frases clave o puntuaciones resumidas.
- ▶ Extracción de entidades: identificar y estructurar nombres, ubicaciones, fechas, productos, síntomas, etc.
- ▶ Conversión texto → variable: Traducir instrucciones o condiciones textuales en una variable computable o categórica.

Ejemplo: a partir de una nota clínica como «Paciente con antecedentes de hipertensión y síntomas de fatiga», el LLM puede generar nuevas columnas: diagnóstico: hipertensión, síntoma principal: fatiga, riesgo: moderado.

En la práctica, este proceso se implementa mediante prompting estructurado y el resultado se valida automáticamente o con revisión asistida. Los valores generados se integran como columnas adicionales en el dataframe original, aumentando su valor predictivo y analítico.

A continuación, otro ejemplo con LLM, incorporando dos variables semánticas a partir de texto: sentimiento (–1/0/1) e intención de compra posterior (baja/media/alta).

Partimos del mismo conjunto de datos de 20 filas y añadimos una columna de texto libre (nota). En un escenario real, estas notas provienen de canales operativos: como mensajes de chat o correo electrónico del cliente, llamadas transcritas del centro de contacto, el campo «observaciones» en CRM/ERP, formularios de devoluciones o incidencias o comentarios de encuestas posventa. El LLM devuelve un JSON con ese par de campos y lo fusionamos al marco de datos para su uso inmediato en análisis o modelado. Usamos temperatura: 0 y una respuesta con un esquema JSON para hacer el proceso estable y fácil de validar.

Código:

```
import os, io, json
import pandas as pd
from openai import OpenAI

# 0) Cliente y modelo (temperatura 0 → resultados estables)
# Pega tu API key en la línea de abajo, reemplazando "sk-x"
client=OpenAI(api_key="sk-x")
MODEL = "gpt-4o-mini-2024-07-18" # sustituir por cualquier modelo compatible con JSON Schema

# 1) Dataset (20 filas) + notas (texto libre)
raw = """"edad,ingresos,compras
22.88,50348.64,5.9
28.7,47742.81,6.52
23.16,46040.51,7.47
20.24,55145.16,8.84
26.2,51920.1,7.02
29.7,55600.22,6.5
32.88,64374.39,11.57
28.5,54913.55,5.74
```

```
29.21,52387.55,6.04
29.59,62575.92,13.05
33.89,61349.92,7.9
23.95,31406.49,6.6
33.76,49881.46,6.51
31.29,54051.98,8.28
36.35,57548.44,9.24
24.46,43829.79,8.95
30.31,50936.34,8.78
31.39,55833.21,9.47
33.09,47977.67,8.64
21.99,41609.94,4.46
""
```

```
df = pd.read_csv(io.StringIO(raw))
```

```
notas = [
    "Pedido con retraso, necesito entrega hoy si es posible. Muy urgente.",
    "Satisfecho con el producto; consulto si hay descuento por volumen.",
    "La app falló al pagar; soporte no respondió. Solicito solución.",
    "Quiero devolver el artículo; llegó dañado y deseo reembolso.",
    "Producto correcto, pero preferiría recoger en tienda el viernes.",
    "Reclamo factura con CIF para empresa; prioridad media.",
    "Excelente calidad, repetiré compra. Recomendaría programa de fidelización.",
    "Demora en el envío otra vez; estoy molesto. Urgente respuesta.",
    "Consulta sobre garantía extendida y reparación en caso de fallo.",
    "Muy contento; entrega rápida por web. Volvería a comprar.",
    "No encuentro el manual; ¿hay versión digital? Gracias.",
    "Cobro duplicado de 39€; exijo abono inmediato.",
    "Me llamaron ayer; acuerdo de cambio para mañana por la tarde.",
    "Quiero aplicar cupón; ¿promoción de verano sigue activa?",
    "Atención correcta, pero el color no coincide; considerar cambio.",
    "Necesito factura electrónica hoy para contabilidad.",
    "Descontento con la atención; me pasaron de un área a otra.",
    "¿Pueden reservar stock? Viajo el lunes y lo necesito listo.",
    "Valoro positivamente el embalaje; sin incidencias.",
    "No volveré a comprar si se repite el retraso."
]
df["nota"] = notas
```

2) Esquema JSON cerrado (solo 2 campos)

```
json_schema = {
    "name": "enriquecimiento_semantico_simple",
    "strict": True,
    "schema": {
        "type": "object",
        "properties": {
            "sentimiento": {"type": "integer", "enum": [-1, 0, 1]},
            "intencion_compra": {"type": "string", "enum": ["baja", "media", "alta"]}
        },
        "required": ["sentimiento", "intencion_compra"],
        "additionalProperties": False
    }
}
```

3) Prompt compacto

```
def build_input(nota: str) -> str:
    return (
        "Devuelve SOLO un JSON que cumpla estrictamente el esquema. "
        "Define 'sentimiento' (-1 negativo, 0 neutro, 1 positivo). "
        "Define 'intencion_compra' (baja|media|alta). "
        f"Texto: \"{nota}\""
    )
```

4) Llamada al LLM por fila (temperature=0, schema estricto)

```
def enriquecer_nota(nota: str) -> dict:
    resp = client.responses.create(
        model=MODEL,
        input=build_input(nota),
        temperature=0,
        response_format={"type": "json_schema", "json_schema": json_schema},
    )
    return json.loads(resp.output_text) # el esquema garantiza JSON válido
```

5) Ejecutar y unir

```
registros = [enriquecer_nota(n) for n in df["nota"]]
df_sem = pd.DataFrame(registros)
df_out = pd.concat([df, df_sem], axis=1)
```

6) Guardar y mostrar

```
out_csv = "datos_20_semantico_sent_intencion.csv"
df_out.to_csv(out_csv, index=False)
print("Guardado en:", out_csv)
print(df_out.head(6)[["nota", "sentimiento", "intencion_compra"]])
```

Estas nuevas columnas se integran al dataframe original para priorizar casos y mejorar el poder predictivo de modelos posteriores. Como resultado, tendremos un dataframe enriquecido con variables semánticas, listas para análisis y modelado. Vemos cómo el LLM transforma cada nota en dos señales operativas: «sentimiento» (tensión o satisfacción percibida) e «intencion_compra» (predisposición comercial). Al integrar estas columnas, se podría, por ejemplo, priorizar (p. ej., atender primero «sentimiento = -1»), segmentar y predecir (compras, retención), comparando modelos con y sin estas variables para cuantificar su impacto predictivo.

Estos son un par de registros de ejemplo con las nuevas variables («sentimiento», «intencion_compra») añadidas por el LLM. Como se puede ver, el tono del texto se refleja en «sentimiento» (negativo o positivo) y la predisposición del cliente en «intencion_compra» (baja, media o alta).

Edad	Ingresos	Compras	Nota	Sentimiento	Intención compra
22.88	50348.64	5.90	Pedido con retraso, necesito entrega hoy si es posible. Muy urgente.	-1	Media
29.59	62575.92	13.05	Muy contento; entrega rápida por web. Volvería a comprar.	1	Alta

Tabla 4. Ejemplo de dos registros con variables semánticas. Fuente: elaboración propia.

En el primer caso, el cliente expresa retraso y urgencia, por eso aparece sentimiento = -1 (negativo) e intención de compra = media (quiere resolver, pero está molesto). En el segundo, el cliente está satisfecho y dice que volvería a comprar; de ahí, sentimiento = 1 e intención de compra = alta.

3.5. Integración de datos externos con IA

Más allá de procesar el texto proporcionado, los LLM avanzados pueden actuar como agentes autónomos, capaces de conectarse con fuentes externas de información para ampliar y enriquecer conjuntos de datos con información actualizada o complementaria. Este enfoque extiende las fronteras del análisis a través de la integración dinámica de datos provenientes de APIs, documentos, bases abiertas o herramientas especializadas.

Un agente de IA es una arquitectura donde un LLM actúa como componente central de razonamiento. Este agente puede seguir un ciclo razonar-actuar (como en el framework ReAct), tomando decisiones paso a paso sobre qué acción ejecutar, qué herramienta utilizar y cómo interpretar la respuesta. Entre las herramientas más comunes están:

- ▶ APIs externas: servicios web con datos en tiempo real, como APIs de clima, precios financieros, tráfico, legislación, enciclopedias o catálogos de productos.
- ▶ Buscadores o motores de recuperación: permiten al agente obtener contexto adicional a partir de documentación técnica, artículos o páginas web estructuradas.

Cuando el agente consulta una API, lo hace mediante una petición estructurada (usualmente en formato JSON) que especifica qué datos necesita. La respuesta recibida puede procesarse automáticamente y utilizarse para rellenar, completar o enriquecer columnas del conjunto de datos.

Como ejemplo, podemos plantear que un analista financiero utilice un agente con acceso a una API de cotizaciones bursátiles. A partir de un listado de empresas, el agente completa columnas como valor actual, variación en la última semana o riesgo sectorial en función de los datos en tiempo real obtenidos de fuentes externas verificadas.

Dentro de las ventajas del enfoque, encontramos:

- ▶ Actualización automática de variables contextuales o externas.
- ▶ Contextualización enriquecida basada en hechos o registros reales.
- ▶ Integración multimodal: el LLM puede razonar sobre la fuente, limpiar la respuesta y transformarla, según necesidades analíticas.

En conjunto, el enriquecimiento semántico y la integración de datos externos convierten a los LLM en herramientas versátiles que no solo analizan lo que ya está presente en los datos, sino también amplían activamente el universo informativo disponible. Esto permite pasar de conjuntos de datos estáticos a sistemas analíticos dinámicos.

3.6. Evaluación del impacto del enriquecimiento

El preprocesamiento y el enriquecimiento de datos mediante IA generativa no constituyen un fin en sí mismos, sino que deben evaluarse en términos de su impacto directo en el rendimiento de los modelos de Machine Learning. Esta evaluación garantiza que las nuevas variables generadas o imputadas aporten información relevante y no introduzcan redundancia, ruido, o incluso, distorsión en el modelo predictivo.

Cada variable incorporada, debe aportar señal predictiva y no solo aumentar la dimensionalidad del dataset. Las variables sintéticas mal calibradas pueden inducir sobreajuste, colinealidad artificial o sesgos implícitos. Por ello, el enriquecimiento se somete a una validación formal para comprobar su valor estadístico y práctico (Gupta *et al.*, 2025).

El enfoque más estandarizado para evaluar el valor de nuevas variables es comparar el rendimiento de un modelo con y sin enriquecimiento. Esto permite calcular el «uplift» o la mejora relativa. El procedimiento incluye:

- ▶ Entrenar un modelo base (sin enriquecimiento).
- ▶ Entrenar un modelo equivalente con las nuevas variables generadas.
- ▶ Comparar ambas versiones sobre un conjunto de prueba estratificado e independiente.
- ▶ Medir las diferencias en métricas relevantes según la tarea y la tabla cinco.

Tipo de tarea	Métricas de evaluación relevantes
Clasificación binaria	<i>Accuracy, Precision, Recall, F1-score, ROC-AUC, PR-AUC, Brier Score.</i>
Regresión	<i>MSE, RMSE, MAE, R².</i>
Segmentación o <i>clustering</i>	<i>Silhouette Score, Calinski-Harabasz, Davies-Bouldin (antes y después del cambio).</i>
Modelos probabilísticos	<i>Log loss, Calibration Error, Reliability diagrams.</i>
<i>Scoring</i>	<i>Lift chart, Cumulative gain, Top decile lift.</i>

Tabla 5. Métricas relevantes según tarea. Fuente: elaboración propia.

Ejemplo: dataset de tickets de soporte, cada uno con una descripción en texto del problema reportado por el usuario. El objetivo es predecir si un caso será resuelto en menos de cuarenta y ocho horas.

Las variables generadas son:

- ▶ **Resumen_1lm:** una síntesis semántica de la descripción del problema generada mediante un LLM (por ejemplo, «GPT-4»), que resume la intención y gravedad del caso.
- ▶ **Tfidf_extra:** un vector de características adicionales derivadas del mismo texto, generado por el método TF-IDF («term frequency-inverse document frequency»), que pondera las palabras más representativas del contenido textual.

TF-IDF es una técnica clásica de minería de texto que transforma el texto en variables numéricas y captura qué términos son relevantes para cada documento en relación con el corpus general. Al combinar esta técnica con resúmenes generados por LLM, se aprovechan tanto las frecuencias estadísticas como la comprensión contextual del lenguaje.

Los resultados comparativos, por ejemplo, de aplicar un modelo de bosque aleatorio se muestran en la tabla 6.

Modelo	Accuracy	F1-score	ROC-AUC
Sin enriquecimiento	0.79	0.75	0.83
Con resumen_ilm	0.85	0.82	0.90
Con resumen_ilm + tfidf_extra	0.86	0.83	0.91

Tabla 6. Ejemplo métricas según modelo propuesto. Fuente: elaboración propia.

La incorporación de variables derivadas del texto no estructurado logra, en el ejemplo, una mejora significativa en la capacidad predictiva del modelo; esto demuestra que el enriquecimiento semántico aporta información relevante y no redundante.

Adicionalmente, se debe realizar una validación complementaria que incluye:

- ▶ Pruebas de ablación: eliminación selectiva de las variables enriquecidas para medir su contribución.
- ▶ Importancia de variables: valores SHAP, importancia por permutación.
- ▶ Curvas de aprendizaje: permiten observar cómo varía el rendimiento del modelo en función del tamaño del conjunto de entrenamiento, lo que revela si las variables enriquecidas son útiles de forma consistente o solo aportan valor en determinadas escalas de datos.
- ▶ Robustez y explicabilidad: validar si la variable generada es estable ante cambios leves en la entrada y si puede interpretarse por un experto.

La evaluación rigurosa del enriquecimiento es esencial para distinguir entre transformaciones informativamente válidas y aquellas que simplemente aumentan la complejidad. Un enfoque basado en uplift, que combine validación empírica, interpretabilidad y robustez, es clave para una práctica profesional y responsable de la IA generativa en ciencia de datos avanzada.

3.7. Automatización del preprocesamiento

Todas las técnicas exploradas hasta este punto pueden integrarse en pipelines de preprocesamiento automatizados, facilitando así su ejecución sistemática y reproducible en entornos de producción o investigación avanzada.

Un pipeline de preprocesamiento es una secuencia estructurada de pasos que transforma los datos en bruto en un formato listo para el modelado. Estos pasos pueden incluir:

- ▶ Imputación de valores faltantes.
- ▶ Normalización o estandarización.
- ▶ Codificación de variables categóricas.
- ▶ Generación de variables sintéticas.
- ▶ Enriquecimiento semántico con LLM.
- ▶ Reducción de dimensionalidad; entre otros.

Los beneficios de un pipeline automatizado son claros: garantiza la reproducibilidad, la trazabilidad, un menor riesgo de errores humanos y la escalabilidad cuando se trabaja con grandes volúmenes de datos o múltiples proyectos simultáneos.

Los LLMs pueden contribuir activamente a esta automatización no solo generando nuevas variables o resúmenes semánticos, sino también escribiendo el código en Python o SQL, que implementa todo el pipeline. Esta capacidad transforma al LLM en un copiloto de datos, permitiendo a los analistas diseñar flujos de trabajo completos mediante lenguaje natural.

Por ejemplo, un analista podría indicar:

«Crea un pipeline que rellene los nulos en «edad» con un VAE, genere una variable de resumen de texto en «comentario», y estandarice todas las variables numéricas».

Y el modelo generará automáticamente un script reproducible con bibliotecas como scikit-learn, SDV, Transformers o pandas, y que integra tanto pasos clásicos como capacidades generativas avanzadas.

La automatización se extiende aún más mediante agentes de IA basados en LLM, capaces de interactuar con herramientas externas a través de API. Un agente es un sistema que combina un modelo de lenguaje con capacidad de razonamiento y acción. Funciona dentro de un marco como ReAct —reason + act—, siguiendo ciclos de decisión que implican:

- ▶ Interpretar una necesidad del usuario.
- ▶ Planificar un conjunto de pasos.
- ▶ Llamar a funciones o herramientas específicas (por ejemplo, API).
- ▶ Recuperar y procesar datos.
- ▶ Generar una salida útil.

Una API (interfaz de programación de aplicaciones) es un mecanismo estandarizado para que dos sistemas se comuniquen. Por ejemplo, una API puede entregar en formato JSON datos meteorológicos, precios financieros, registros clínicos anonimizados o bases de datos académicas. Un agente inteligente puede consultar una API, interpretar su respuesta y enriquecer automáticamente un conjunto de datos en curso.

Con herramientas como LangChain, PandasAI o DSPy, es posible construir agentes basados en LLM que integran dinámicamente múltiples fuentes y herramientas dentro de un pipeline automatizado. Estos frameworks actúan como capas de orquestación que permiten a los modelos de lenguaje no solo interpretar instrucciones en lenguaje natural, sino también ejecutar código, interactuar con API externas, manipular estructuras de datos (como DataFrames) y coordinar decisiones complejas paso a paso.

- ▶ LangChain se especializa en conectar LLM con herramientas externas (navegadores, API, bases de datos, funciones definidas por el usuario) para construir agentes complejos que razonan y actúan en múltiples etapas del flujo de trabajo.
- ▶ PandasAI es una biblioteca de Python que permite a un LLM operar directamente sobre dataframes de pandas: formulando consultas, resúmenes o transformaciones sin escribir código manual. Es ideal para tareas exploratorias y de limpieza.
- ▶ DSPy ofrece una arquitectura modular para diseñar programas dirigidos por LLM con componentes reutilizables, validación automática y pasos de inferencia personalizados. Esto facilita la creación de flujos de trabajo robustos y evaluables.

Estas herramientas convierten al agente no solo en un generador de código, sino también en un ejecutor, capaz de tomar decisiones condicionales, adaptar la lógica según el contenido de los datos y asegurar que el flujo de trabajo responda a requisitos dinámicos. De este modo, se materializa un paradigma en el que el preprocesamiento deja de ser una secuencia fija de funciones para convertirse en un sistema cada vez más autónomo, flexible y alineado con los objetivos analíticos.

Este enfoque híbrido, en el que LLM, agentes inteligentes y las API trabajan de forma coordinada, representa una transformación radical del preprocesamiento tradicional. En particular, los flujos de trabajo generativos automatizados con LLM son ideales para tareas bien definidas, estructuradas y repetibles, como la limpieza, el

enriquecimiento o la imputación. Por su parte, los agentes de IA son más adecuados para flujos dinámicos, contextualizados y dependientes de condiciones externas, como API en tiempo real. Ambos enfoques son complementarios en una arquitectura moderna de preprocesamiento.

3.8. Exploración práctica

En esta sección se presentan dos casos aplicados que integran las técnicas abordadas a lo largo del tema, desde la imputación hasta la automatización mediante LLM y el acceso dinámico a API externas. Cada caso refleja escenarios en los que se aprovecha el potencial de la IA generativa para enriquecer y transformar conjuntos de datos reales de forma reproducible y escalable.

Caso Práctico 1: Enriquecimiento semántico y evaluación de impacto

Una empresa de telecomunicaciones desea predecir qué clientes tienen un alto riesgo de cancelar su contrato (churn). Dispone de datos básicos de uso y de una columna de texto libre con la «última queja o consulta» del cliente.

Objetivo: evaluar si esta información semántica puede mejorar la predicción del churn model.

Datos

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# Datos simulados en memoria (muestra realista para pruebas)
data = {
    "id_cliente": [f"C-{i:03d}" for i in range(1, 11)],
    "antiguedad_meses": [3, 24, 1, 36, 2, 48, 5, 18, 1, 60],
    "consumo_gb": [50, 15, 5, 25, 10, 30, 60, 20, 8, 40],
    "llamadas_soporte": [4, 1, 2, 0, 3, 1, 5, 0, 2, 1],
    "ultima_queja": [
        "La velocidad de internet es terrible, nunca llega a lo prometido.",
        "Todo funciona perfectamente, muy contento con el servicio.",
        "Me han cobrado de más en la factura de este mes.",
        "Sin problemas hasta la fecha.",
        "El técnico nunca llegó a la cita para la instalación.",
        "El servicio es fiable y la atención al cliente siempre ha sido buena.",
    ]
}
```

```

    "La conexión se cae constantemente, es imposible teletrabajar.",
    "Funciona bien.",
    "Quiero cancelar mi contrato, el servicio es pésimo.",
    "Llevo años con ustedes y la calidad siempre ha sido excelente."
],
"churn": [1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
}
telecom_df = pd.DataFrame(data)

```

Enriquecimiento semántico con LLM

```

# DataFrame que simula la respuesta del LLM para cada cliente
enriquecimiento_simulado = pd.DataFrame({
    "id_cliente": [f"C-{i:03d}" for i in range(1, 11)],
    "sentimiento_queja": ["Negativo", "Positivo", "Negativo", "Positivo", "Negativo",
"Positivo", "Negativo", "Positivo", "Negativo", "Positivo"],
    "tema_principal": ["Velocidad Internet", "Sin Queja", "Facturación", "Sin Queja",
"Servicio Técnico", "Sin Queja", "Velocidad Internet", "Sin Queja", "Servicio Técnico", "Sin
Queja"],
    "intencion_cancelar": [False, False, False, False, False, False, False, False, True,
False]
})
telecom_df = telecom_df.merge(enriquecimiento_simulado, on="id_cliente")

```

Evaluación de Impacto Predictivo

```

# Codificación de variables categóricas
telecom_encoded = pd.get_dummies(telecom_df, columns=["sentimiento_queja",
"tema_principal"], drop_first=True)

# Definición de las características para cada modelo
features_baseline = ["antiguedad_meses", "consumo_gb", "llamadas_soporte"]
features_enriquecidas = features_baseline + [col for col in telecom_encoded.columns if
'sentimiento_queja_' in col or 'tema_principal_' in col]
features_enriquecidas.append('intencion_cancelar')

# Separación de datos
X_baseline = telecom_encoded[features_baseline]
X_enriquecido = telecom_encoded[features_enriquecidas]
y = telecom_encoded["churn"]

Xb_train, Xb_test, y_train, y_test = train_test_split(X_baseline, y, test_size=0.3,
random_state=42, stratify=y)
Xe_train, Xe_test, _, _ = train_test_split(X_enriquecido, y, test_size=0.3,
random_state=42, stratify=y)

```

```
# Entrenamiento y evaluación de los modelos
modelo_base = RandomForestClassifier(random_state=42).fit(Xb_train, y_train)
modelo_enriquecido = RandomForestClassifier(random_state=42).fit(Xe_train, y_train)

print("--- INFORME DE RENDIMIENTO: MODELO BASE (SIN ENRIQUECIMIENTO) ---")
print(classification_report(y_test, modelo_base.predict(Xb_test), zero_division=0))

print("\n--- INFORME DE RENDIMIENTO: MODELO ENRIQUECIDO CON IA
GENERATIVA ---")
print(classification_report(y_test, modelo_enriquecido.predict(Xe_test), zero_division=0))
```

Aplicación

Al ejecutar el código, se observa un salto dramático en el rendimiento. El F1 para la clase 1 (churn) en el modelo base probablemente será bajo (p. ej. 0,00 o 0,50, dependiendo de la división), mientras que en el modelo enriquecido será perfecto o casi perfecto (1,00). Esto indica que las nuevas variables semánticas (sentimiento_queja_negativo, intencion_cancelar, etc.) no son solo «ruido», contienen una señal predictiva extremadamente fuerte.

La mejora no es únicamente un número. En un escenario real, significa que el modelo enriquecido es significativamente mejor para identificar correctamente a los clientes que están en riesgo real de abandonar el servicio. Esto permite al equipo de retención actuar de forma proactiva y dirigida. Este ejercicio demuestra cómo el enriquecimiento con IA generativa puede tener un ROI (retorno de la inversión) directo y medible.

Caso práctico 2: pipeline automatizado con agente LLM + API externa real

Un analista en una aseguradora de salud recibe cada semana un conjunto de datos de consultas de pacientes.

Objetivo: Construir un pipeline automático que:

Imputar valores nulos en la edad.

Utilizar un agente LLM para extraer semántica clínica del texto libre.

Consultar la API externa real Open-Meteo para añadir contexto ambiental.

Datos

```
import pandas as pd
import numpy as np
```

```
# Datos base con valores nulos
data_salud = pd.DataFrame({
    "id": ["P001", "P002", "P003", "P004", "P005"],
    "edad": [35, 60, np.nan, 22, 75],
    "sexo": ["F", "M", "F", "M", "F"],
    "ciudad": ["Madrid", "Barcelona", "Valencia", "Sevilla", "Bilbao"],
    "consulta": [
        "Dolor torácico agudo y dificultad severa para respirar.",
        "Chequeo general sin síntomas aparentes.",
        "Tos persistente y fatiga desde hace tres semanas.",
        "Lesión en la rodilla jugando al fútbol.",
        "Seguimiento de hipertensión, se siente mareada."
    ]
})

# Simulación del enriquecimiento semántico del LLM
enriquecido_semantico = pd.DataFrame({
    "id": ["P001", "P002", "P003", "P004", "P005"],
    "clasificacion_riesgo": ["Alto", "Bajo", "Medio", "Bajo", "Medio"],
    "sospecha_diagnostico": ["Angina/Embolia Pulmonar", "Sin hallazgos",
        "Bronquitis/Neumonía", "Lesión de ligamentos", "Crisis hipertensiva"]
})
```

Integración con API

```
import requests

def obtener_aqi_para_ciudades(ciudades: dict):
    url = "https://air-quality-api.open-meteo.com/v1/air-quality"
    resultados = []
    print("Consultando la API de Open-Meteo para obtener datos de calidad del aire...")

    for ciudad, coords in ciudades.items():
        lat, lon = coords
        params = {"latitude": lat, "longitude": lon, "current": "european_aqi"}

        try:
            response = requests.get(url, params=params)
            response.raise_for_status()
            data = response.json()
            aqi = data.get('current', {}).get('european_aqi')

            if aqi is not None:
                resultados.append({"ciudad": ciudad, "aqi_ciudad": aqi})
                print(f"Éxito para {ciudad}: AQI = {aqi}")
            else:
                resultados.append({"ciudad": ciudad, "aqi_ciudad": None})

        except requests.exceptions.RequestException as e:
            print(f"Error al consultar la API para {ciudad}: {e}")
            resultados.append({"ciudad": ciudad, "aqi_ciudad": None})

    return pd.DataFrame(resultados)

# Definimos las coordenadas de las ciudades
ciudades_a_consultar = {
    "Madrid": (40.4168, -3.7038), "Barcelona": (41.3851, 2.1734),
    "Valencia": (39.4699, -0.3763), "Sevilla": (37.3891, -5.9845),
    "Bilbao": (43.2630, -2.9350)
}

# Llamamos a la función para obtener los datos reales
df_aqi_real = obtener_aqi_para_ciudades(ciudades_a_consultar)
```

Pipeline completo

```
from sklearn.impute import SimpleImputer

def pipeline_salud_completo(df_crudo, df_semantico, df_api):
    df = pd.merge(df_crudo, df_semantico, on="id")
    df = pd.merge(df, df_api, on="ciudad")
    imputer = SimpleImputer(strategy="mean")
    df["edad"] = imputer.fit_transform(df[["edad"]])
    df.drop(columns=["consulta", "id"], inplace=True)
    return df

# Aplicación del pipeline
df_listo_para_modelar = pipeline_salud_completo(data_salud, enriquecido_semantico,
df_aqi_real)
print("\n--- DataFrame Final Enriquecido y Listo para Modelar ---")
print(df_listo_para_modelar)
```

Aplicación

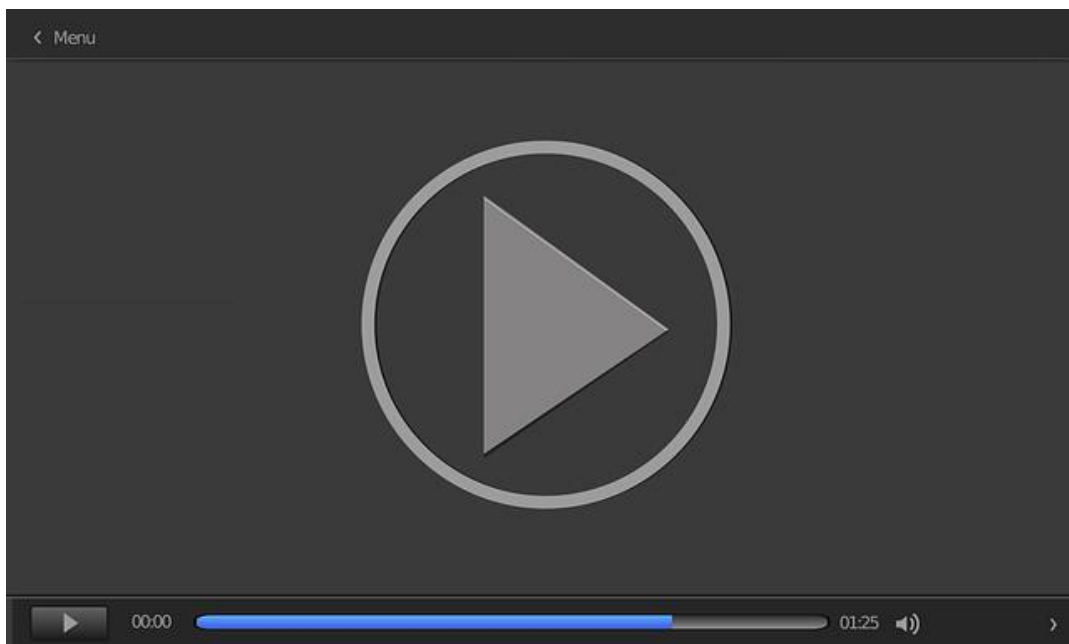
Este caso práctico ya no es una simulación: demuestra un flujo de trabajo de extremo a extremo que un analista de datos realizaría en un entorno profesional, combinando datos internos, el enriquecimiento por la IA y la ingesta de datos de una fuente externa en tiempo real.

La capacidad de integrar API es una habilidad fundamental. Este ejercicio explica el concepto y presenta el código práctico para hacerlo.

El DataFrame final es inmensamente más rico que el original. Ahora es posible construir modelos que no solo consideren los síntomas del paciente (*sospecha_diagnostico*), sino también su contexto ambiental (*aqi_ciudad*). Esto podría revelar correlaciones importantes —p. ej., si las consultas por problemas respiratorios aumentan en ciudades con peor calidad del aire—.

Durante la elaboración del tema se empleó un modelo de IA generativa como apoyo para optimizar la redacción y precisar conceptos técnicos. Su uso se llevó a cabo con responsabilidad y criterio académico mediante múltiples iteraciones de consulta, análisis y reformulación que, con la debida experticia, permitieron ajustar las respuestas con rigor y precisión en el contexto específico del contenido desarrollado (OpenAI, 2025). Este uso complementario no sustituyó la consulta de fuentes académicas ni el ejercicio intelectual de construcción, sino que buscó reforzar la calidad y la claridad en el marco de una práctica académica transparente y ética.

En el vídeo, *Enriquecimiento semántico y sintético con IA generativa*, muestra cómo enriquecer conjuntos de datos con variables generadas a partir de texto, imputaciones y descripciones automáticas. Incluye ejemplos con datos reales y visualización del antes y después.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=064c3558-48fa-4837-adcc-b377013cfa1b>

3.9. Referencias bibliográficas

LangChain. (2025). *LangChain documentation*.
<https://python.langchain.com/docs/introduction/>

Página de ChatGPT <https://chat.openai.com/>

Página de OpenAI. developer platform <https://platform.openai.com/docs/overview>

Ramchandani, T. (2025). *A generative journey to AI: Mastering the foundations and frontiers of generative deep learning*. BPB Publications.

Scikit-learn (2025). Sklearn.impute. IterativeImputer. <https://scikit-learn.org/stable/modules/generated/sklearn.impute.IterativeImputer.html>

Yoon, J., Jordon, J. y van de Schaar, M. (2018). GAIN: missing data imputation using generative adversarial nets. *Proceedings of the 35th International Conference on Machine Learning*, 80, 5689-5698. <https://proceedings.mlr.press/v80/yoon18a.html>

Paper: GAIN

Yoon, J., Jordon, J. y van der Schaar, M. (2018). GAIN: Missing data imputation using generative adversarial nets. *Proceedings of the 35th International Conference on Machine Learning*, 80, 5689–5698. <https://arxiv.org/abs/1806.02920>

Proporciona la profundidad teórica para entender la imputación de datos. No solo describe el **qué**, sino que explica el **por qué** y el **cómo** de la imputación adversarial, detallando la arquitectura del modelo, su fundamento matemático y el innovador mecanismo de **pista** que lo hace efectivo. Apoya el conocimiento para entender, evaluar y, potencialmente, innovar sobre las técnicas de imputación más recientes que existen.

Tutorial: LangChain

LangChain. (2025). *LangChain Documentation*.
<https://python.langchain.com/docs/introduction/>

La documentación de LangChain es valiosa porque actúa como un puente entre la teoría conceptual de los LLM y su aplicación práctica en el código. Su importancia radica en que muestra, a través de sus tutoriales, cómo forzar salidas estructuradas (como JSON), encadenar tareas de preprocesamiento en pipelines robustos y construir agentes que interactúan con API externas, en *scripts* de Python funcionales y reproducibles.